

Smart MCQ Solver Challenge - Project Report

BSDA2001P : Introduction to DL and GenAI Project

Roll No. – 23f3002289

1. Executive Summary

This project focuses on the **Smart MCQ Solver Challenge**, an AI competition where the goal is to predict the top three most likely answers (from options A to E) for science and reasoning multiple-choice questions. The models are evaluated using MAP@3, a metric that awards higher scores when the correct answer is ranked at or near the top.

To solve this problem, three different approaches were tested and compared:

- **Model 1 (Custom Deep Neural Network):** A custom-built neural network that reads questions and options converted into word and character pattern counts (TF-IDF features).
- **Model 2 (RNN Sequence Model):** A Bidirectional LSTM network with an attention mechanism designed to read sentences in both directions and highlight key context words.
- **Model 3 (Pretrained Embedding Lookup):** A simple retrieval system using a pre-trained BAAI language model to find similar past questions without any extra training.

Tracking the results on Weights & Biases showed that models specifically trained to evaluate question-option pairs where Model 1 and Model 2 performed significantly better than Model 3.

2. Introduction

This project is based on the Kaggle **Smart MCQ Solver Challenge**, where each question has five options (A–E) and only one correct answer. The goal is to predict the **top 3 most likely answers in ranked order**, using MAP@3 as the evaluation metric.

The main objective of this project was to build and compare **three different models**: one built from scratch, one using a pretrained model without training, and one additional deep learning approach. This helped me compare different techniques for solving the same MCQ problem.

3. Dataset and Preprocessing

3.1 Dataset Description

The dataset contains two files: **train.csv** and **test.csv**. The training file contains the correct answer for each question. Each row has a question prompt and five answer options, **A, B, C, D and E**.

3.2 Exploratory Data Analysis (EDA)

Before building the models, I checked the dataset for missing values, answer distribution and duplicate records. There were **no missing values** in the dataset.

I also checked for duplicate questions along with their five answer options. **183 duplicate question-option sets** were found in the training data, leaving **1,817 unique question-option sets** out of 2,000 rows. These duplicates were considered while creating the training and validation splits to reduce the possibility of data leakage.

3.3 Text Preprocessing

The preprocessing was different for each model:

1. **TF-IDF + DeepNet:** The prompt was combined separately with each option using prompt + [SEP] + option. The resulting text pairs were converted into numerical TF-IDF features using word and character n-grams.
2. **BGE:** The prompt and all five options were combined into one text and converted into a semantic embedding using the pretrained BGE model.
3. **BiLSTM + Attention:** The text was converted to lowercase, tokenized into words, mapped to numerical token IDs and padded to a fixed sequence length.

4. Tokenization Strategy

For TF-IDF + DeepNet, TF-IDF converts the text directly into sparse numerical vectors using word and character n-grams. For BGE, the tokenizer built into the Sentence-Transformer model is used, and the complete question is converted into a dense embedding.

For BiLSTM + Attention, the text is converted to lowercase and tokens are extracted using the pattern [a-z0-9]+. A vocabulary is then created from the question and option text, with <pad> used for padding and <unk> for words not present in the vocabulary. Each sequence is limited to 60 tokens and padded to the same length.

5. Modelling & Experimentation

5.1 Model 1: TF-IDF + Custom Feed-Forward Neural Network (DeepNet)

The first model was my traditional baseline. I created five prompt-option pairs for every question. The correct option was assigned label 1 and the other four options were assigned label 0. This turns the problem into binary scoring for each option. After getting five scores, I ranked them and selected the top three.

I used two TF-IDF representations. The word-level vectorizer used unigrams and bigrams, English stop-word removal, a maximum of 50,000 features and sublinear TF. The character-level vectorizer used character n-grams from 3 to 5 characters with a maximum of 30,000 features.

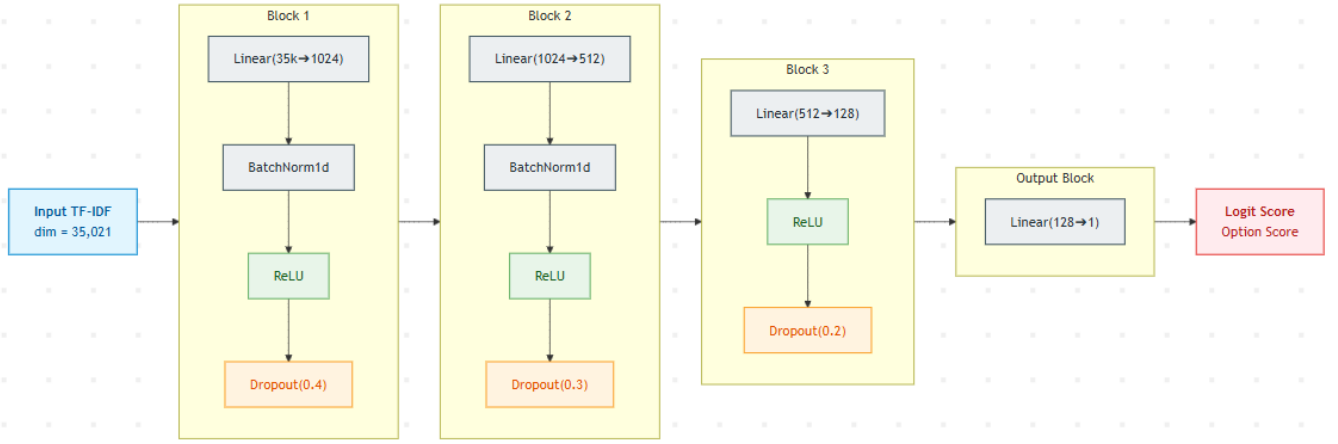


Figure 1 : Custom Feed-Forward Neural Network Architecture

5.2 Model 2: Pretrained BGE Model

The second model uses **BAAI/bge-small-en-v1.5**, a BERT-style encoder-only Sentence-Transformer model that generates a dense embedding for each input text. For each question, the **prompt and all five options are joined using [SEP]** and passed through BGE to obtain a dense, L2-normalized embedding. The embedding of a validation or test question is then compared with the embeddings of the training questions using cosine similarity, and the **20 most similar training questions** are retrieved. Their known answer labels are combined using a **similarity-weighted voting approach**, and the three options with the highest weighted scores are selected as the final top-3 predictions.

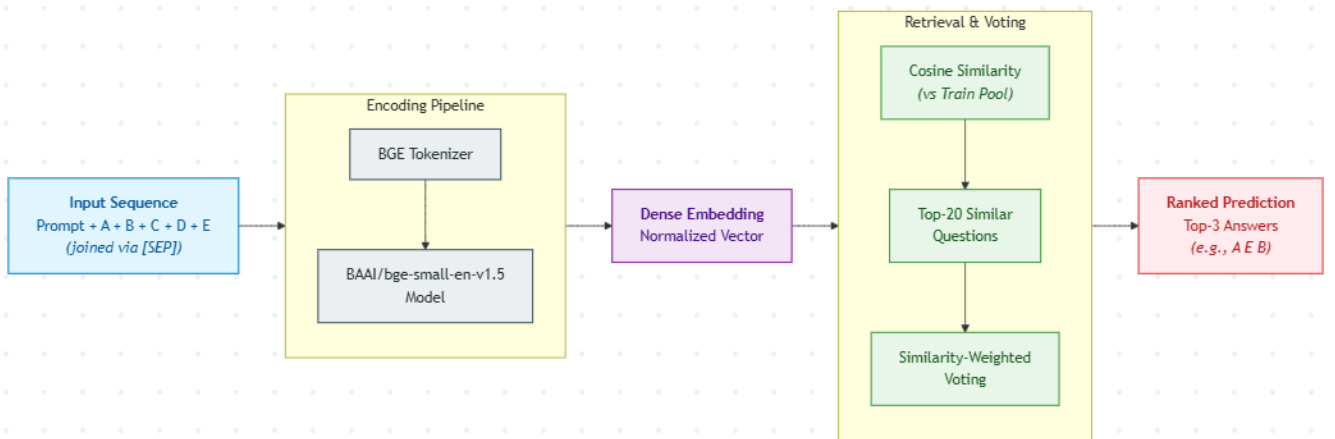


Figure 2 : Pretrained BGE Model Architecture

5.3 Model 3: BiLSTM with Attention-Based Scoring

The third model uses a **BiLSTM with an attention mechanism** to learn the relationship between the prompt and each answer option. The token IDs are first converted into **128-dimensional embeddings** and passed through a bidirectional LSTM with a hidden size of 128, giving a **256-dimensional representation** at each timestep.

An attention layer then assigns a score to each timestep, while padding positions are masked before applying softmax. The weighted LSTM outputs are combined to form a fixed-length representation. The same encoder is used for the prompt and all five options, so they share the same weights. For each prompt-option pair, the prompt and option representations are combined using the two vectors, their absolute difference, and element-wise multiplication.

This combined representation is passed through a small feed-forward scoring network of $1024 \rightarrow 256 \rightarrow 128 \rightarrow 1$, producing one score for each option. The five scores are then ranked to obtain the top-3 predictions.

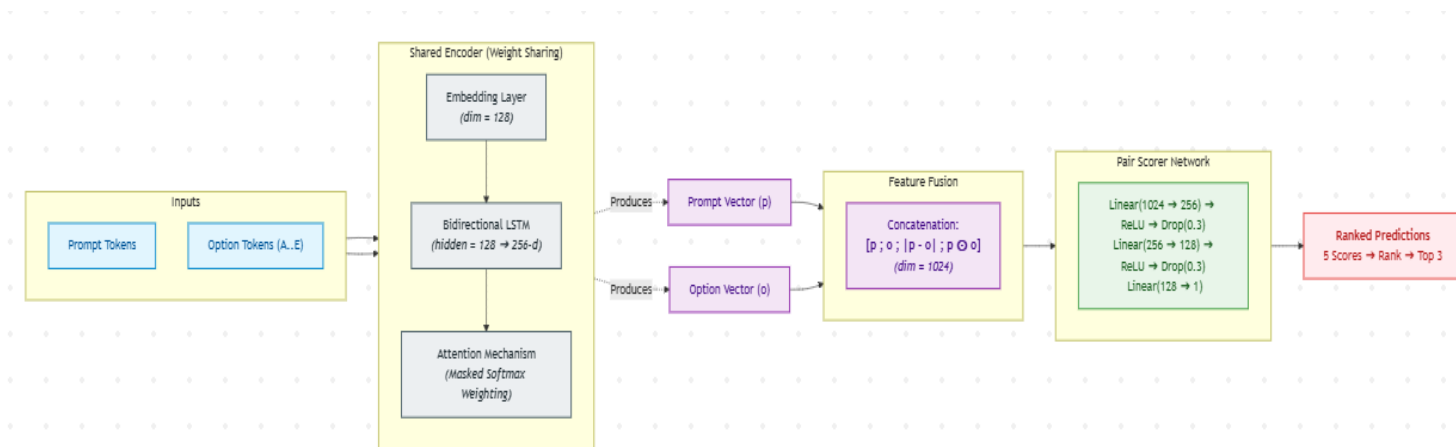


Figure 3 : BiLstm model Architecture

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

MAP@3 is the main competition metric. For each question, the correct answer receives a score of 1 if it is ranked first, 1/2 if it is ranked second and 1/3 if it is ranked third. If it is outside the top three, the score is 0. The final MAP@3 is the average over all questions.

I also used Accuracy and Macro F1 during validation. Accuracy measures how often the first prediction is correct. Macro F1 calculates F1 separately for each answer class and then gives all five classes equal weight.

6.2 Comparative Report

Model Name	Accuracy	F1- Score	MAP@3
Tfidf+Deepnet	0.99500	0.99499	0.99750
Pretrained BGE	0.77250	0.77352	0.88083
Bilstm	0.99725	0.99719	0.99863

6.3 Training Behavior

The DeepNet model reached a high MAP@3 very quickly. It reached 0.9800 in the first epoch and 0.9975 in the second epoch. After that, MAP@3 remained at 0.9975 while validation loss continued to move up and down. Early stopping finally stopped the run after three consecutive non-improving validation-loss checks.

The BiLSTM showed a clearer learning curve. Its MAP@3 improved from 0.7092 to 0.9986 over seven epochs. After epoch 7 the validation score stayed almost unchanged, so the model had effectively reached a stable point for this split.

BGE is different because there is no training curve. It is a retrieval-only experiment. The model is loaded once, embeddings are created and the answer is predicted from nearest training examples.

6.4 Kaggle Performance

The final competition submission achieved a Kaggle leaderboard score of **0.74355 MAP@3**

7. Conclusion & Future Work

7.1 Key Learnings

- TF-IDF can be surprisingly strong when both word and character features are combined.
- A BiLSTM with attention can learn the relationship between a question and its answer choices directly.
- MAP@3 is important for this competition because ranking the second or third option can still receive useful credit.
- A very high validation score does not always mean the model will perform equally well on hidden test data.
- Using different model types helped me understand the difference between sparse features, pretrained embeddings and learned sequence representations.

7.2 Challenges faced

One of the main challenges was handling five options for every question and ranking them correctly. The TF-IDF model required converting each question into multiple prompt-option pairs, while BGE required generating and comparing embeddings for the training questions. The BiLSTM required additional steps such as vocabulary creation, tokenization and padding. Another major challenge was the gap between the high validation scores and the final **0.74355 MAP@3** Kaggle score, which showed the difficulty of generalizing to unseen questions.

7.3 Areas of Improvement

1. **Model Ensemble:** Combine the predictions of TF-IDF, BGE and BiLSTM to improve ranking reliability.

2. **Fine-Tuning:** Fine-tune a pretrained transformer model on the MCQ dataset to better adapt its representations to the specific question-answering task.
3. **Better Semantic Models:** Explore stronger pretrained embedding or transformer models to capture deeper semantic relationships between questions and options.

8. References

1. Scikit-learn documentation — TfidfVectorizer and evaluation metrics.
2. PyTorch documentation — neural networks, LSTM, optimization and training.
3. Sentence-Transformers documentation — sentence embeddings and similarity search.
4. BAAI/bge-small-en-v1.5 — pretrained BGE sentence embedding model.
5. Weights & Biases documentation — experiment tracking and metric logging.